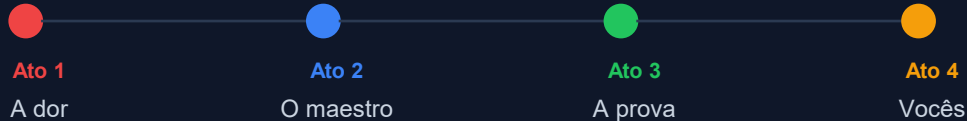
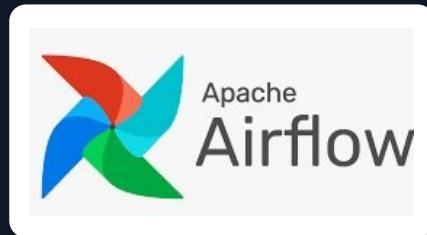


Data Integration e Pipelines

Aula 4 — Apache Airflow, Kubernetes e Orquestração



Prof. Rafael S Novo Pereira
FIAP — MBA Data Engineering



ATO 1 · SLIDES 3–6

A dor

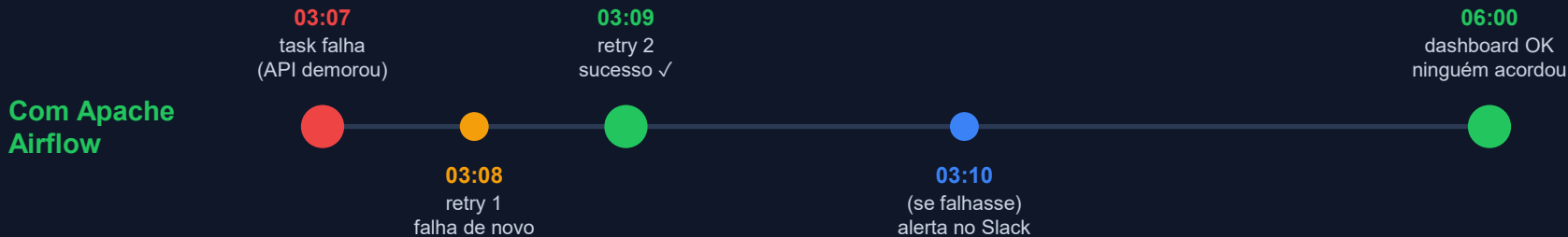
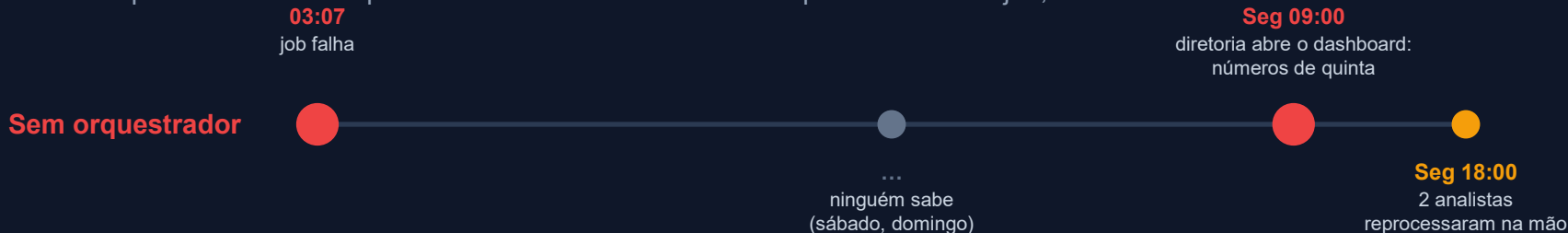
Por que um pipeline que "funciona" ainda não está em produção.

Vocês vão sair deste ato sabendo:

- ✓ Por que apertar play não escala — e quanto custa quando falha às 3h
- ✓ O que Databricks, Snowflake e dbt resolveram, e o buraco que ficou
- ✓ O que muda no dia a dia com um orquestrador

Sexta-feira, 3h07: a jornada de uma falha

Cenário hipotético construído para a aula — não é um incidente reportado. Mesmo job, duas realidades.



Mesma falha, mesmo job, mesmo dado. A diferença é quem está acordado às 3h07: uma pessoa — ou um orquestrador.

As cinco perguntas — e os cinco nomes

Cada pergunta do cenário tem uma resposta com nome próprio no Airflow

1 Quem deveria ter rodado o job às 3h?

`schedule`

2 Quem deveria ter tentado de novo às 3h08?

`retry`

3 Quem deveria ter acordado alguém às 3h10?

`alerta`

4 Como reprocessar sexta, sábado e domingo sem reescrever código?

`backfill`

5 Como provar que o dado de segunda está certo ANTES de servir?

`quality check`

No fim da aula vocês vão ter visto os cinco funcionando de verdade — não em slide, num pipeline rodando.

A jornada do curso: 4 aulas, 4 ferramentas

Cada aula resolveu um problema — e revelou o próximo



Aula 1

Databricks

Bronze/Silver/Gold
em PySpark + Delta

× tudo manual



Aula 2

Snowflake

SQL, Tasks, Streams
Azure Blob externo

× só dentro do Snowflake



Aula 3

dbt

Modelos versionados
Testes de qualidade

× só transforma



Aula 4 — hoje

Airflow

Orquestra tudo:
GitHub → Snowflake → dbt

VOCÊ ESTÁ AQUI

Cada ferramenta é excelente no que faz. Nenhuma, sozinha, resolve o fluxo completo. A Aula 4 existe para dar sentido às outras três.

Sem orquestrador vs com Apache Airflow

O que muda na segunda-feira de manhã

Sem orquestrador

- ✗ Alguém aperta play manualmente
- ✗ Falha às 3h — ninguém sabe
- ✗ Sem retry automático
- ✗ Log espalhado: notebook, worksheet, console
- ✗ Cada ferramenta numa ilha
- ✗ "Funciona no meu notebook"

Com Apache Airflow

- ✓ Pipeline roda no schedule definido em código
- ✓ Alerta se falhar (email, Slack, Astro Alerts)
- ✓ Retry automático: `retries=2`, `retry_delay=1min`
- ✓ Log de cada task, de cada execução, no browser
- ✓ DAG define ordem e dependências entre ferramentas
- ✓ Orquestra Snowflake, dbt, APIs, GitHub, Databricks

O maestro

O que é o Apache Airflow, quem usa, e como ele funciona por dentro.

Vocês vão sair deste ato sabendo:

- ✓ O que é uma DAG — e por que ela é uma receita com ordem
- ✓ Fan-out e fan-in: como o Airflow roda coisas em paralelo
- ✓ A vida de uma task: os estados que vocês vão ver na Grid
- ✓ As 5 peças do Airflow 3 e por que o Astro esconde todas elas

Quem usa Airflow?

Fonte: lista oficial [apache/airflow INTHEWILD.md](#) — 583 organizações autodeclaradas (consultada em 08/10/2026)

Brasil

- Banco Inter
- Stone
- Creditas
- QuintoAndar
- EBANX
- 99
- Paraná Banco
- Ministério da Economia

Mundo

- Airbnb (criador)
- Spotify
- PayPal
- Reddit
- Tesla
- Bloomberg
- Adobe
- Zalando

Saber Airflow hoje é como saber SQL há 10 anos: não é diferencial — é pré-requisito.

Apache Airflow em números

Verificados em 08/10/2026 — cada número diz quem o contou

30M+

downloads / mês

Astronomer (fornecedor,
autodeclarado)

3.6K+

contribuidores

Astronomer (fornecedor,
autodeclarado)

~80K

organizações

anúncio Airflow 3.0 (abr/2025),
estimativa

3.3

versão do lab

Astro Runtime 3.3-6 · 3.3.0 em
06/07/2026 · última: 3.3.2



2014

nasce no Airbnb



2015

open source



2019

top-level Apache



abr/2025

Airflow 3.0
(reescrita: UI nova, isolamento)



jul/2026

Airflow 3.3
(o do lab)

Airflow não processa dados. Ele orquestra quem processa.

O que é uma DAG?

Directed Acyclic Graph — grafo direcionado sem ciclos. Analogia: uma receita de bolo



Receita de bolo

1. Separar ingredientes
2. Misturar a massa (depende de 1)
3. Colocar na forma (depende de 2)
4. Assar 40 min (depende de 3)
5. Testar com o palito (depende de 4)
6. Servir (depende de 5)



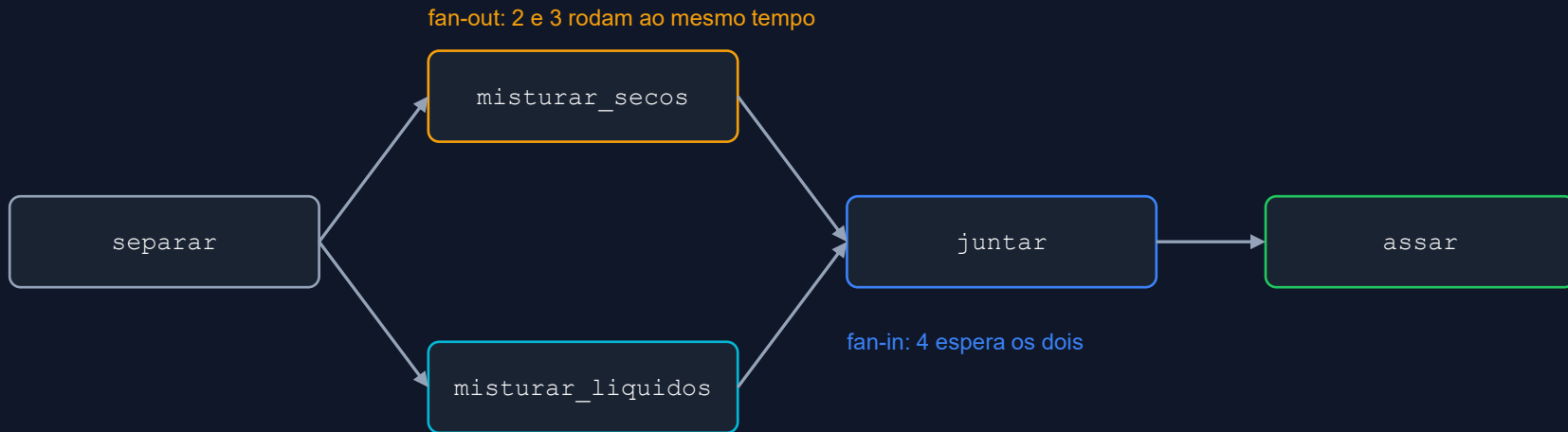
DAG do lab: pipeline_olist_completo

1. inicio
2. `download_and_load_bronze` (depende de 1)
3. `transform_silver` (depende de 2)
4. `build_gold` (depende de 3)
5. `quality_checks` (depende de 4)
6. fim (depende de 5)

DAG = receita com ordem. Não pode assar antes de misturar. Não pode transformar antes de carregar. Se o palito falha, não serve.

Paralelismo: fan-out e fan-in

A receita de verdade não é linear — e a DAG também não precisa ser



```
separar >> [misturar_secos, misturar_liquidos] >> juntar >> assar  
# lista = paralelo · '>>' = depende de · o scheduler decide quantos workers usar
```

A jornada de uma task: os estados da Grid

Cada cor da interface do Airflow é um estado desta jornada

Quando um quadrado ficar vermelho no lab, vocês vão saber exatamente em que ponto da jornada ele parou — e o log diz por quê.

Arquitetura do Airflow 3: quem move a task pela jornada

Analogia: uma cozinha profissional — os componentes que aparecem na tela de saúde do Astro



API Server

O cardápio

Interface (UI) e API. Onde vocês olham, acionam e limpam.



Scheduler

O chef

Olha relógio e dependências. Decide o que vai para a fila.



DAG Processor

Quem lê a receita

Lê os .py da pasta dags e monta o grafo.



Workers

Os cozinheiros

Executam as tasks. No Astro: um pod K8s por task.



Metadata DB

O caderno

Estado de cada task, histórico, conexões.

+ Triggerer: segura tasks que esperam evento (deferred) sem ocupar worker.

No Airflow 2 o worker falava direto com o banco. No Airflow 3 tudo passa pelo API Server e o worker não enxerga o Metadata DB — é isso que permite rodar a task em qualquer lugar.

Por que estamos usando o Astro?

ASTRONOMER

A plataforma gerenciada da Astronomer para Apache Airflow



Airflow na nuvem

Não instala nada. Abre o browser e usa. Deployment HEALTHY em 2–5 min, com as 5 peças do slide anterior prontas.



Kubernetes por baixo

Cada task roda num pod isolado. Escala sozinho. Vocês não configuram K8s — mas vão ver a prova de que ele está lá.



Trial gratuito

14 dias, US\$ 20 em créditos, sem cartão (visto na tela de signup em ago/2026 — pode mudar; confira na semana da aula).



Git integrado

Conecta o repositório. "Deploy Project" constrói a imagem Docker, sobe no cluster e registra a DAG.

Astro é para o Airflow o que o Snowflake é para SQL: a versão gerenciada, sem dor de cabeça com infraestrutura.

Astro: o que vocês vão ver

The screenshot displays the Astro web interface. At the top, a teal banner indicates a trial status: "You have \$20 in credits and 14 days left in your trial." with an "Explore Plans" button. On the left is a dark purple sidebar with the Astro logo and navigation links: "WORKSPACE Fiap", "Home", "Deployments", "DAGs", "Astro IDE" (with a "Preview" button), "Environment", and "Workspace Settings". Below these are "ORGANIZATION" links, including "Get Started with Astro!" which is highlighted with a progress bar at 0/3. At the bottom of the sidebar is a user profile icon and a help icon. The main content area is titled "Get Started with Astro" and features a numbered guide:

- 1 Create an Airflow Deployment**
An Astro Deployment is an Airflow environment that is powered by Astro Runtime. It runs all core Airflow components, including the Airflow webserver, scheduler, and workers, plus additional tooling for reliability and observability.
 - Create a Deployment** (RECOMMENDED)
The quickest way to create a Deployment is to use the Astro UI.
[Create Deployment](#)
 - Explore a Demo**
Tour an established Astro Organization to see the value of data connections, alerting, and reporting.
[Start Product Demo](#)
- 2 Create and run DAGs**
- 3 Manage Your Environment**

Below the guide is a section for "Helpful Resources".

Tela inicial do Astro após criar a conta — Etapa 5 do Guia Visual. O botão verde "Create Deployment" é o primeiro clique.

A prova

Um pipeline real: 100 mil pedidos brasileiros, do GitHub ao Snowflake, em Kubernetes.

Vocês vão sair deste ato sabendo:

- ✓ A jornada do dado: 9 CSVs (7 usados) → Bronze → Silver → Gold → validação, e quem faz cada passo
- ✓ A DAG em Python — o arquivo real do repositório
- ✓ Schedule, retry, timeout, alerta e backfill funcionando (e um deles ao vivo)
- ✓ Por que cada task rodou num container — com a prova de que eu quebrei o pipeline por causa disso

A jornada do dado: Olist, do GitHub ao dashboard

Olist — e-commerce brasileiro, dados públicos 2016–2018, ~100 mil pedidos · tempos da execução real



● cheio = dado parado (tabela)

○ vazio = task (um pod K8s)

■ bronze / prata / ouro = camada

Total: 1 min 49 s. Ninguém abre o Snowflake. Os mesmos dados Olist da Aula 3 — agora com maestro.

O código: uma DAG em Python

dags/pipeline_olist.py — trecho real do repositório, sintaxe Airflow 3 (airflow.sdk)

```
from airflow.sdk import dag, task
from airflow.providers.standard.operators.empty import EmptyOperator
from datetime import datetime, timedelta

@dag(
    dag_id="pipeline_olist_completo",
    schedule=None,          # manual — trocamos depois
    start_date=datetime(2025, 1, 1),
    catchup=False,          # NÃO reprocessar desde 2025
    default_args={
        "retries": 2,
        "retry_delay": timedelta(minutes=1),
        "execution_timeout": timedelta(minutes=30)},
    tags=["fiap", "olist"],
)
def pipeline():
    inicio = EmptyOperator(task_id="inicio")
    fim     = EmptyOperator(task_id="fim")

    @task
    def download_and_load_bronze(): ... # pandas + write_pandas
    # transform_silver, build_gold, quality_checks: idem

    (inicio >> download_and_load_bronze() >> transform_silver()
     >> build_gold() >> quality_checks() >> fim)

pipeline()
```

Python puro

Sem XML, sem YAML, sem arrastar caixinha.
Versionado no Git, revisado em pull request.

default_args

retry, retry_delay e timeout valem para todas as tasks. Três das cinco respostas do cold open, em três linhas.

>> define a ordem

Se download falha, silver não roda. Se quality_checks falha, fim não roda — e o dashboard não recebe dado ruim.

Schedule: quando o maestro levanta a batuta

O agendamento está no código — versionado no Git, revisado em pull request

```
schedule=None
```

Manual — aperta play (nosso lab, primeira execução)

```
"@daily"
```

Todo dia à meia-noite UTC (preset — cuidado com o fuso)

```
"0 6 * * *"
```

Todo dia às 6h · cron = minuto hora dia mês dia-da-semana

```
"*/5 * * * *"
```

A cada 5 minutos (bom para demo)

```
"0 22 * * TUE,THU"
```

Terça e quinta às 22h

```
timedelta(hours=2)
```

A cada 2 horas, contadas a partir do start_date

```
Asset("../pedidos.csv")
```

Quando o dado chega — orientado a evento (Airflow 3)

EXERCÍCIO (em dupla): o financeiro pede o fechamento no primeiro dia ÚTIL de cada mês, às 7h. Escrevam o schedule. Dica: pensem no que acontece quando o dia 1 cai no sábado.

O que acontece quando o pipeline falha?

As quatro respostas do Airflow — três já estão no código do lab



Retry automático

```
retries=2, retry_delay=timedelta(minutes=1)
```

API caiu? Tenta de novo, sozinho. Estado `up_for_retry` → `queued`. ✓ no lab



Timeout e Deadline

```
execution_timeout=timedelta(minutes=30)
```

Demorou demais? Mata a task. Deadline Alerts (Airflow 3.1+) avisam ANTES de estourar. O "SLA" do Airflow 2 foi removido no 3.0. ✓ timeout no lab



Alertas

```
on_failure_callback / Astro Alerts
```

Alguém precisa saber. Email exige SMTP; no Astro, Alerts de UI para email/Slack/PagerDuty. X não configurado — desafio Prata C



Backfill

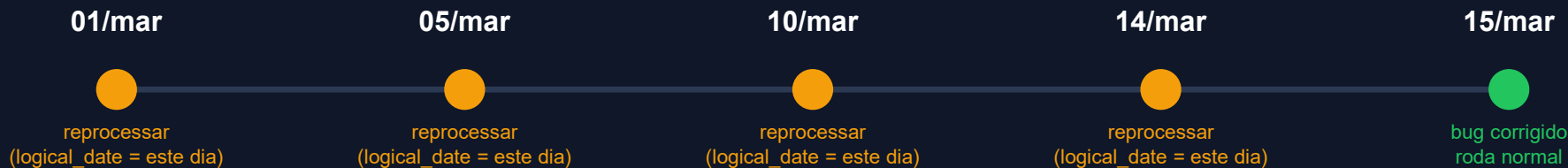
```
schedule + logical_date + botão Backfill
```

Reprocessar o passado sem reescrever código. Exige `schedule` ≠ `None` e task que lê a data. X na DAG principal — desafio Prata B

Backfill: a máquina do tempo dos dados

Cenário hipotético para a aula — não é um incidente reportado

Imaginem: um banco digital descobre em 15/mar que o cálculo de score de crédito está errado desde 01/mar. O código foi corrigido hoje. E os 14 dias de score errado que já viraram decisão de crédito?



Um comando de backfill de 01/mar a 14/mar: o Airflow cria 14 execuções, uma por dia, cada uma com o logical_date daquele dia. A task usa essa data para filtrar o que baixa, transforma e valida. Zero linha de código nova.

EXERCÍCIO: um banco descobre em 20/ago que o pipeline diário de compliance está errado desde 01/jan. (a) Quantas execuções de backfill? (b) O que o dashboard mostra ENQUANTO reprocessa — e o que vocês fariam para o regulador não ver número intermediário? (c) Que pré-condição a task precisa ter para o backfill ser seguro?

Demo ao vivo: backfill em 90 segundos

Três mudanças na DAG + um botão na interface do Airflow 3 — dags/demo_backfill.py

```
@dag(
    dag_id="demo_backfill",
    schedule="@daily",          # 1. tem schedule
    start_date=datetime(2026, 1, 1), # 2. no passado
    catchup=False,
)
def demo():
    @task
    def que_dia_eu_sou(**ctx):
        d = ctx["logical_date"] # 3. a task lê a data
        print(f"Processando o dia {d:%Y-%m-%d}")
        que_dia_eu_sou()
    demo()
```



O que a sala vai ver

1. Abro a DAG demo_backfill
2. Trigger ▾ → Backfill
3. Últimos 7 dias, sem reprocessar existentes
4. Sete execuções aparecem na Grid
5. Abro o log de uma: "Processando o dia ..."

O que levar: a task não sabe "hoje". Ela sabe logical_date. Escrever a task em função dessa data é o que torna o backfill possível — e é o que a maioria dos pipelines escritos em notebook não faz.

Kubernetes: por que o Airflow roda em containers



Isolamento

Cada task = 1 container. A biblioteca da task 2 não conflita com a da task 3.



Escalabilidade

Dez tasks em paralelo? Dez pods. Uma? Um pod.



Eficiência

O pod nasce, roda a task e morre. Você paga só o que usa.



Resiliência

Pod crashou? K8s detecta, Airflow marca falha e faz retry noutro pod.

No Astro isso é automático — vocês não configuram Kubernetes, não escrevem YAML, não usam kubectl. Mas precisam saber o que acontece por baixo, porque isso explica o comportamento da task (os segundos que ela demora para começar) e a conta no fim do mês.

Astro: vocês já estão usando Kubernetes

A jornada do código até o pod — e a prova de que cada task é um container isolado



Seu código (.py)



Imagem Docker



Cluster Kubernetes



1 pod por task



Airflow UI (browser)



A prova real (aconteceu construindo este lab)

Separei download e load em duas tasks. A task de download gravou o CSV em /tmp. A task de load não encontrou o arquivo: /tmp não existia — porque era OUTRO pod, com OUTRO filesystem. Solução: uma única task download_and_load_bronze. Isso é Kubernetes funcionando.

Tudo isso acontece com um clique em "Deploy Project". O Astro constrói a imagem, sobe no cluster e registra a DAG. Vocês nunca veem o cluster — mas ele está lá.

Pipeline real: todas as tasks verdes

Dag: ☒ pipelineolist_completo

Execução do Dag: manual_2026-03-26T21:04:12.354661+00:00

Opções

00:02:53

00:01:26

início

download_and_load_bronze

transform_silver

build_gold

quality_checks

fim

download_csvs_github

load_bronze_snowflake

manual_2026-03-26T21:04:12.354661+00:00 ✓ Sucesso

Adicionar uma nota

Limpar Execução do Dag

Marcar Execução do Dag como...

Data Lógica: 2026-03-26 18:04:10

Tipo de Execução: manual

Data Inicial: 2026-03-26 18:04:13

Data Final: 2026-03-26 18:06:02

Duração: 00:01:49

Nome do Usuário que Disparou: cmn7ncimw00bp01muremo7pdk

Versão(s) do Dag: v2

Instâncias de Tarefa

Asset Events

Log de Auditoria

Código

Detalhes

Pesquisar Tarefas

Todos os Estados

6 Instâncias de Tarefa

ID da Tarefa	Índice do Mapa	Estado	Data Inicial	Número de Tentativas	Operador	Duração
fim		✓ Sucesso	2026-03-26 18:06:01	1	EmptyOperator	
quality_checks		✓ Sucesso	2026-03-26 18:05:55	1	PythonOperator	00:00:05.551
build_gold		✓ Sucesso	2026-03-26 18:05:47	1	PythonOperator	00:00:06.030
transform_silver		✓ Sucesso	2026-03-26 18:05:38	1	PythonOperator	00:00:07.249
download_and_load_bronze		✓ Sucesso	2026-03-26 18:04:36	1	PythonOperator	00:00:59.830
início		✓ Sucesso	2026-03-26 18:04:13	1	EmptyOperator	

Execução real — Airflow 3.3 em Kubernetes no Astro. início → download_and_load_bronze (59 s) → transform_silver (7 s) → build_gold (6 s) → quality_checks (5 s) → fim. Total ≈ 1 min 49 s.

Resultado: dados reais no Snowflake

1  | SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO LIMIT 5;

2

 Add to Chat |  Explain |  Quick edit |  Format

Results (just now)

Table Chart

5 rows ⓘ 86ms

	ESTADO	# TOTAL_PEDIDOS	# RECEITA_TOTAL	# TICKET_MEDIO	# DIAS_ENTREGA_MEDIO	# PCT_ATRASO	ATUALIZADO_EM
1	SP	41746	5998226.96	143.69	8.7	5.72	2026-03-26 14:05:50.714 -0700
2	RJ	12852	2144379.69	166.85	15.2	12.95	2026-03-26 14:05:50.714 -0700
3	MG	11635	1872257.26	160.92	11.9	5.48	2026-03-26 14:05:50.714 -0700
4	RS	5466	890898.54	162.99	15.2	6.99	2026-03-26 14:05:50.714 -0700
5	PR	5045	811156.38	160.78	11.9	4.88	2026-03-26 14:05:50.714 -0700

```
SELECT * FROM OLIST_LAB.PUBLIC.GOLD_RECEITA_ESTADO LIMIT 5;
```

SP lidera com ~R\$ 6 M em 41.746 pedidos. RJ tem 12,95% de atraso. Ninguém tocou no Snowflake.

O que torna um pipeline robusto?

O checklist que separa amador de profissional — e onde o nosso lab está hoje



✓ Idempotência

Rodar 2× = mesmo resultado. Bronze com overwrite, Silver/Gold com CREATE OR REPLACE.



✓ Retry

retries=2, retry_delay=1 min em default_args.



✓ Quality checks

Gold vazia ou receita negativa? A task falha e o fim não roda.



✓ Observabilidade

Log por task, duração, histórico na Grid. Métricas de negócio ficam para vocês.



X Alertas

Não configurado: exige SMTP ou Astro Alerts. → Desafio Prata C



X Backfill

schedule=None e carga total por execução. → Desafio Prata B / Ouro

4 de 6. Os outros 2 são o desafio de vocês — e é assim na vida real: ninguém entrega os 6 no dia 1.

ATO 4 · SLIDES 29–37

Vocês

Do exercício ao lab, dos desafios ao pitch — e o que vocês levam daqui.

Vocês vão sair deste ato sabendo:

- ✓ Desenhar uma DAG de negócio do zero (companhia aérea)
- ✓ Subir o pipeline Olist no seu próprio Astro, com o seu Snowflake
- ✓ Escolher o desafio Prata/Ouro que prova o que faltou no checklist
- ✓ Preparar o pitch de consultoria que fecha a disciplina

Panorama: ferramentas de orquestração

Quem sabe Airflow aprende as outras em dias — o conceito é o mesmo

Airflow	Python, DAGs, extensível, maior comunidade, padrão de mercado	Curva de aprendizado; infra própria (ou Astro/MWAA/Composer)
Prefect	Python nativo, deploy simples, boa DX	Comunidade menor; menos presente em vagas
Dagster	Orientado a assets, tipagem forte, ótimo com dbt	Mais recente; modelo mental diferente
dbt platform (jobs)	Schedule nativo para projetos dbt	Só orquestra dbt — não ingere, não chama APIs
Snowflake Tasks	Nativo, sem ferramenta externa (Aula 2)	Só dentro do Snowflake

Exercício: desenhem a DAG

Cenário: companhia aérea — pipeline diário de atrasos (grupos de 3)

O pipeline precisa:

1. Baixar os dados de voos do dia da API da ANAC
2. Baixar os dados de clima do INMET (independente de 1)
3. Cruzar voos com clima — depende de 1 e 2
4. Calcular taxa de atraso por aeroporto
5. Se taxa > 30% em algum aeroporto: gerar alerta
6. Enviar relatório por email — sempre, com ou sem alerta

Entreguem no papel / quadro

- O desenho da DAG (caixas e setas)
- Quais tasks rodam em paralelo?
- Como o email roda mesmo sem alerta?
- Onde entram retry e quality check?
- Bônus: o schedule em cron

Antes do lab: 3 avisos que evitam 40 minutos perdidos



1. MFA no Snowflake

Desde ago/2026 o Snowflake exige MFA para usuários humanos com senha. Se a Connection do Airflow falhar com erro de autenticação/MFA: crie um usuário de serviço (TYPE = SERVICE) com par de chaves — o guia tem o passo. Nunca use sua senha pessoal numa ferramenta.



2. Trial do Snowflake

Dura 30 dias. Se o seu, da Aula 2, expirou: crie um novo trial agora — 3 minutos. Anote o Account Identifier novo (formato ORG-CONTA).



3. Trial do Astro

Dura 14 dias. Crie a conta HOJE, não antes — para que esteja viva na entrega dos desafios. Configure o Wake Schedule para o horário em que vai trabalhar.

Guia Visual Passo a Passo (.docx) — 16 etapas com screenshots. Sigam na ordem. Levantem a mão na Etapa 13.

A jornada do lab: 90 minutos, 6 estações, 2 checkpoints

Guia Visual, Etapas 1–16 · trabalhem em dupla, um Astro por dupla

Etapas 1–5 · 10 min

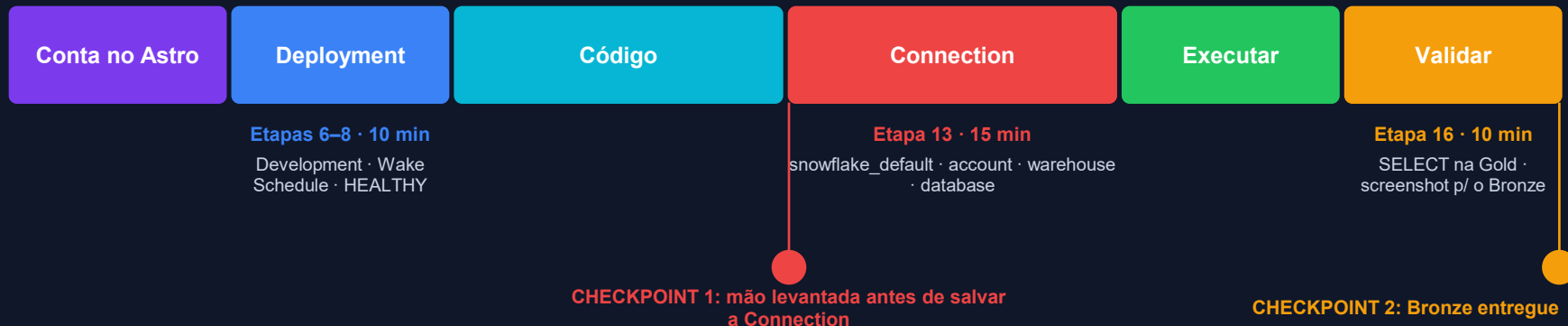
Signup · Organization ·
tela principal

Etapas 9–12 · 15 min

GitHub → Astro IDE → Deploy → DAG
na lista

Etapas 14–15 · 10 min

Trigger → Execução Única
→ 6 verdes



+ 20 min de folga para os erros previstos. Quem terminar antes: desafio Prata.

Os 5 erros mais prováveis, na ordem

Etapa 10.3 "Astro Project Path" preenchido (deixe vazio) · Etapa 13.4 account no formato errado (é ORG-CONTA, sem .snowflakecomputing.com) · Etapa 13 erro de MFA (plano B do guia) · DAG não aparece após deploy (espere HEALTHY, 3–5 min, recarregue) · quality_checks vermelho com warehouse suspenso (re-execute a task)

Desafios: Bronze, Prata e Ouro

Entrega individual pelo portal — PDF com screenshots + link do seu fork · prazo: [DEFINIR]

3 Bronze — obrigatório (em aula)

- Pipeline rodando: Grid com as 6 tasks verdes
- Resultado do SELECT na GOLD_RECEITA_ESTADO
- Log da task download_and_load_bronze mostrando as 7 tabelas

Critério: screenshots legíveis com seu nome de usuário visível. Nota: até 7,0.

2 Prata — escolha 1 (ou 2)

- A. Faça um quality check FALHAR de verdade (ex.: exigir atraso < 10% por estado) — entregue o log da falha e explique por que o "fim" não rodou
- B. schedule="@daily" + task que lê logical_date → backfill de 3 dias, Grid com 3 execuções
- C. Configure um alerta (Astro Alerts ou on_failure_callback) e prove que disparou

Critério: código no seu fork + evidência de execução. Cada Prata: +1,5.

1 Ouro — bônus

- Suba a DAG pipeline_olist_dbt.py (Cosmos + dbt, no mesmo repo): o projeto dbt da Aula 3 orquestrado pelo Airflow, com o lineage visível na UI
- Ou: torne a Bronze idempotente POR DATA (particionada por logical_date) — o pré-requisito real de backfill

Critério: DAG verde + explicação de 5 linhas do que mudou. Ouro: +1,0 (teto 10).

Pitch de consultoria

A atividade que fecha a disciplina: vocês são a consultoria, a turma é o cliente

Formato

- 3 minutos de pitch
- 2 minutos de perguntas da turma
- One-page com: cenário de negócio, diagrama de arquitetura, ferramentas escolhidas e por quê, qualidade e operação (os 6 pilares)
- Databricks E Snowflake obrigatórios; dbt e Airflow contam a favor
- card_transdata.csv não pode ser usado

Critérios

Cenário de negócio	15%
Diagrama de arquitetura	25%
Ferramentas e justificativas	25%
Qualidade e operação	20%
Comunicação e clareza	15%

Parem e pensem: de onde vocês vieram



Quatro semanas atrás, vocês não sabiam o que era uma DAG, um Operator, um backfill, um pod.



Hoje vocês:

- ✓ Construíram pipelines Bronze/Silver/Gold em duas plataformas
- ✓ Transformaram com dbt, com teste de qualidade em cada camada
- ✓ Orquestraram ~100 mil pedidos reais de ponta a ponta — sem apertar play
- ✓ Viram retry, backfill e quality check funcionando — e sabem quais pilares faltam
- ✓ Rodaram tasks em Kubernetes de verdade, sem uma linha de YAML
- ✓ Conseguem explicar num pitch por que o orquestrador é a cola

Isso é mais do que muito engenheiro de dados sênior consegue articular. Vocês estão prontos.

Material e próximos passos



Repositório

github.com/rafsp/MBA_Aula2708

- datasets/ — 9 CSVs do Olist (7 usados)
- dags/pipeline_olist.py — DAG principal
- dags/demo_backfill.py — a demo de hoje
- dags/pipeline_olist_dbt.py — Cosmos + dbt (Ouro)
- dags/dbt/olist/ — projeto dbt da Aula 3
- Dockerfile · requirements.txt · packages.txt
- Guia Visual (.docx) e este deck (.pdf)



Depois da aula

- Entregar Bronze (e Prata/Ouro) pelo portal até [DEFINIR]
- Preparar o pitch de consultoria (one-page + 3 min)
- Hibernar/apagar o Deployment do Astro quando terminar — os créditos são finitos
- Para continuar: Astronomer Academy (cursos gratuitos) e a documentação oficial do Airflow 3

Data Integration e Pipelines

16 horas — o que vocês construíram:

2 plataformas de dados

Databricks (PySpark + Delta) e Snowflake (SQL)

Transformação com dbt

Modelos SQL, testes, documentação, lineage

Orquestração com Airflow 3.3

DAGs, retry, backfill, Kubernetes, operação

Dados reais brasileiros

Olist (~100 mil pedidos) e TPC-H SF1 (~8,6 M linhas)

Kubernetes por baixo

Containers isolados, escaláveis, eficientes